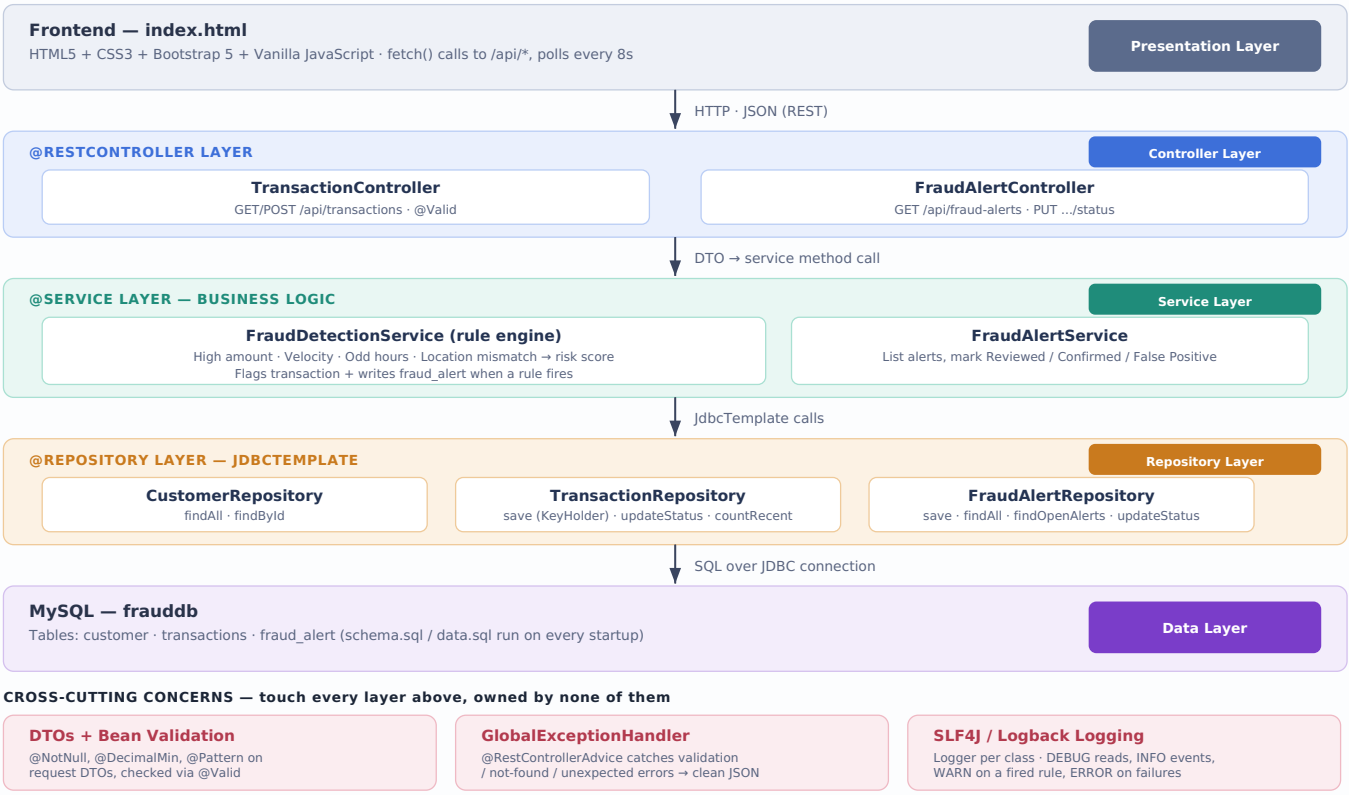
System Component Diagram — MySQL Version

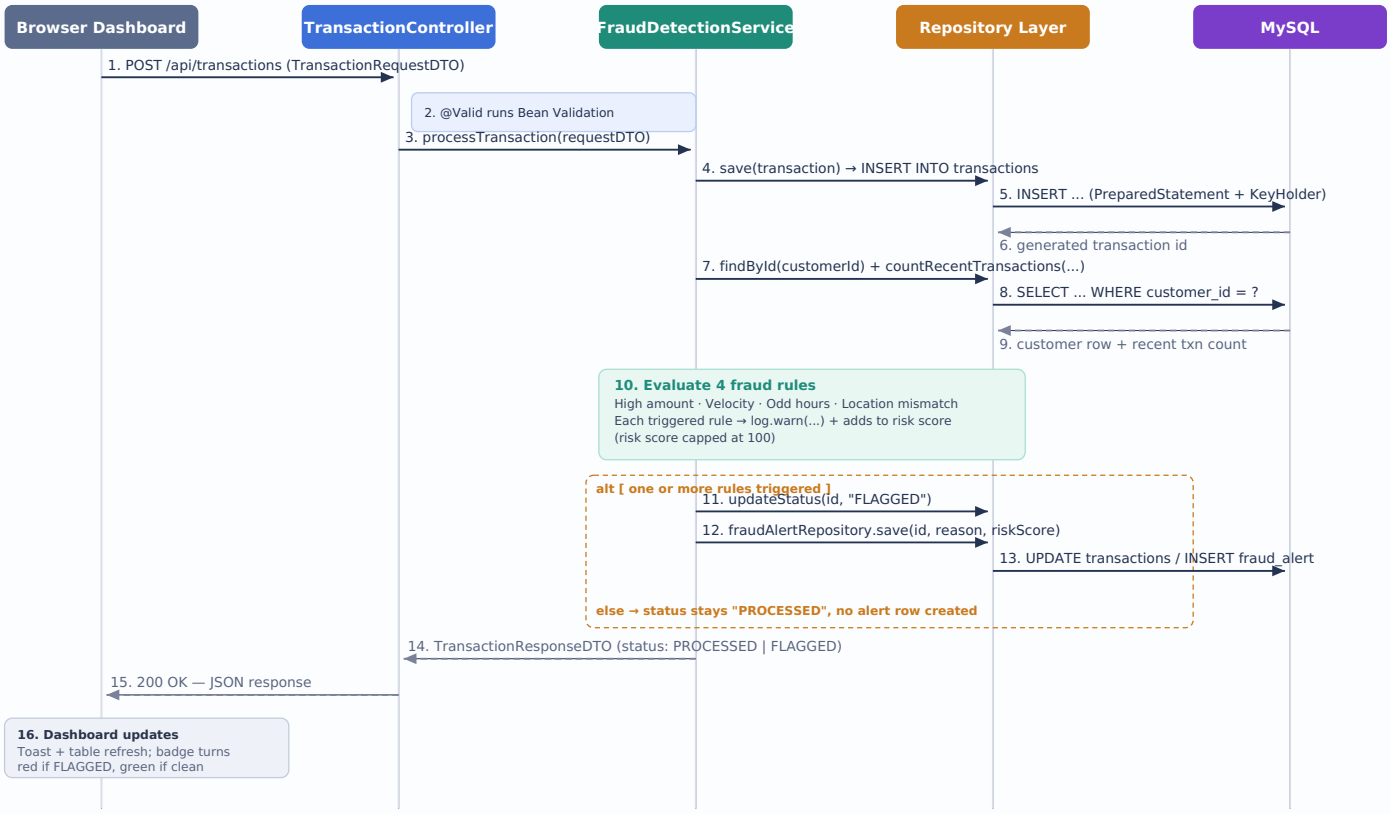
The layered architecture used throughout this project. Each layer has exactly one job — the browser never talks to the database directly, and business rules never live in the controller. Three cross-cutting concerns (bottom row) touch every layer without belonging to any single one of them.



COMPONENT	LAYER	RESPONSIBILITY
index.html	Frontend	Renders the dashboard, submits transactions, polls the API, never talks to MySQL directly.
TransactionController / FraudAlertController	Controller	Exposes REST endpoints, validates incoming DTOs with `@Valid`, delegates everything else to a service.
FraudDetectionService	Service	Owns the four fraud rules, computes the risk score, decides PROCESSED vs FLAGGED.
FraudAlertService	Service	Reads/updates alert records — reviewed, confirmed fraud, or false positive.
*Repository classes	Repository	Hand-written SQL via `JdbcTemplate` — no JPA/Hibernate magic, so freshers see every query.
MySQL (frauddb)	Data	Stores customer, transactions, and fraud_alert tables.
DTOs, Exception Handler, Logging	Cross-cutting	Shape the API contract, turn errors into clean JSON, and make every rule decision traceable in the console.

Request Interaction Flow — Simulating a Transaction

What actually happens, in order, when a user clicks "Submit Transaction" on the dashboard. Solid arrows are calls; dashed arrows are returns. This is the single best diagram to walk through live, one arrow at a time, in your session.



Solid arrow = synchronous call Dashed arrow = return value Dashed box = conditional (alt) branch